PROJECT WALKTHROUGH

BEEJAM

A complete, teacher-ready explanation of the storefront code: what each file contains, how the parts work together, and how a shopper moves through the app.

HTML structure | CSS design | JavaScript behavior | Shopping flow

Project files covered

index.html — 1,213 lines
script.js — 1,696 lines
style.css — 3,960 lines

Prepared from the BEEJAM project files in the workspace. Line numbers refer to the inspected versions of those files.

1. What the project is

BEEJAM is a browser-based demonstration storefront for sustainable everyday products. A visitor can browse categories, search and filter products, open a product detail view, add products to a cart or wishlist, enter delivery details, choose a demonstration payment method, and view an order confirmation.

The project is a front-end prototype. HTML provides the page and controls, CSS controls how they look at different screen sizes, and JavaScript fills product areas and reacts to clicks and form input. The files run together in a browser; no compilation step or server-side application code is present in these three files.

> **Simple presentation:** “HTML builds the storefront, CSS styles it, and JavaScript makes it interactive. The data and shopping state live in JavaScript while the page is open.”

How the files connect

1. The browser opens `index.html`.
2. The HTML head loads `style.css` for layout and design.
3. At the end of the body, `script.js` is loaded.
4. When the document is ready, JavaScript finds elements by their IDs/classes, renders catalog content, and attaches event handlers.
5. CSS classes such as `active-view` and `mobile-open` are toggled by JavaScript to reflect the current interface state.

Important distinction

The source includes login and payment screens, but the project does not connect to a real user account service, inventory database, order server, or payment gateway. Payment success is simulated by browser code. This is appropriate for demonstrating a UI flow, but is not a real commerce system.

2. Project map and reading guide

File / region	Purpose	What to explain
`index.html` lines 1–18	Document setup	Language, mobile viewport, title, description, font and CSS links.
`index.html` lines 19–1212	Storefront markup	Header, mobile navigation, page views, forms, payment overlay, footer and toast container.
`script.js` lines 1–647	Catalog data	Category and product records used to draw cards and details.
`script.js` lines 648–676	Application state and startup	In-memory state and initial rendering.
`script.js` lines 678–1695	Interaction logic	Navigation, product views, cart, filters, checkout, payment simulation and responsive controls.
`style.css` lines 1–1929	Base visual system and desktop layouts	Colors, typography, shared controls, page-specific layouts and overlays.

`style.css` lines 1930–3947	Responsive rules and later UI details	Tablet/mobile layout, login and payment styling, and small-screen overrides.

Line ranges are navigation aids, not a claim that every line is a separate feature. Blank lines and comments divide related rules and functions.

3. HTML: the page skeleton

3.1 Document setup — lines 1–18

`<!DOCTYPE html>` asks the browser to use modern HTML standards mode. `<html lang="en">` declares the page language for assistive technology and language-aware tools. The UTF-8 declaration supports a wide range of characters. The viewport declaration makes layout dimensions follow the device width so the mobile CSS can work properly.

The title is shown in the browser tab and the description provides a short page summary. The favicon points to the BEEJAM logo. Two Google Fonts preconnects prepare the browser to fetch the IBM Plex Serif font; the stylesheet request loads the font weights. Finally, `style.css` is attached to the document.

3.2 Main wrapper and shared header — lines 19–113

The body starts with `.mockup-app-container`, a centered maximum-width canvas. A semantic `<header>` contains the logo link, primary navigation, mobile hamburger, search, wishlist, cart and account buttons. The navigation uses `data-nav` values such as `home` and `shop`; JavaScript reads these values to choose a view. Buttons have accessible labels, while SVG markup draws simple line icons without separate icon image files.

Wishlist and cart badges are spans with stable IDs so JavaScript can update their counts. The hamburger calls `toggleMobileNav()`. Below the header, a hidden mobile drawer and overlay provide a compact navigation pattern on narrow screens.

3.3 Home view — lines 115–197

`#view-home` is the initially visible `.page-view`. It contains the hero message and calls to action, followed by category and featured-product regions. The category and product grid containers have IDs (`home-categories-grid`, `home-featured-grid`) that JavaScript fills from the catalog data. The lower promotional/sustainability content is ordinary HTML and routes into other views.

3.4 Shop/catalogue — lines 199–321

`#view-shop` contains a sidebar with category, price, material and availability controls. The main region contains a search field, sort selector, result count, and product grid. These controls have IDs and names used by `setupEventListeners()`. The product grid starts as a placeholder; JavaScript inserts cards. The pagination visuals are part of the mockup and do not implement a true multi-page data request.

3.5 Product detail page — lines 323–448

`#view-pdp` is a product detail page (PDP). It holds breadcrumbs, a main product image and thumbnail strip, product title, price, rating, descriptive badges, quantity controls, add-to-cart and wishlist buttons, and tab panels for description/material/care/reviews. Related products use a JavaScript-populated grid. Stable IDs allow the selected product data to replace the initial example.

3.6 Cart and checkout — lines 450–636

The cart view has a dynamic item list and a separate summary for subtotal, delivery and total. The checkout view has a progress indicator, contact and delivery fields, and an order summary. The form uses `novalidate`, meaning the browser's built-in validation bubbles are disabled; JavaScript applies the project's own rules and inline messages instead. IDs like `ch-name` and `ch-pincode` connect inputs to the validation rules.

3.7 Success, order details and sustainability — lines 638–785

The success view presents confirmation messaging. The order-details view displays the generated order number, date, items, delivery address, payment method and totals. Sustainability has its own view with four pledges and supporting visual content. In this single-page design, these are sections switched in and out rather than separate HTML documents.

3.8 Login and payment overlays — lines 787–1212

The login panel is an overlay, initially hidden with inline `display:none`; it includes email/password fields and a password-visibility control. The payment overlay contains delivery details, a countdown, an order summary and three tabs: QR, UPI and card. Each tab has a corresponding panel. The page ends with shared toast/notification markup and then loads `script.js`, allowing the HTML elements to exist before the script runs.

HTML patterns used throughout

- **Semantic elements:** `header`, `nav`, `main`, `section` and `form` describe the role of each region.
- **IDs:** unique anchors for JavaScript updates and event handling.
- **Classes:** reusable visual styling hooks, often shared by many elements.
- **Data attributes:** `data-nav` stores a small behavior value on a navigation element.
- **Inline event attributes:** some controls call named functions directly, while other events are attached centrally in JavaScript.
- **Accessibility details:** image alternative text, button labels and navigation labels help communicate purpose.

4. JavaScript: data, state and startup

4.1 Catalog data — lines 1–647

`BeejamData` is a constant object with two main arrays. `categories` contains IDs, display names, counts and images. `products` contains one object per product with fields such as ID, name, category, material key, price, original price, discount, rating, review count, image, thumbnails, description, detailed material/care text and specification labels.

These records are the single source used by category cards, product cards, product detail, cart totals and order details. Product IDs (for example `p1`) connect cart/wishlist entries to their full catalog record. `category` and `materialKey` are compact values for filtering; the matching display labels can be longer.

4.2 Global state — lines 648–663

The `state` object tracks data that changes during a visit: cart entries, last order, wishlist IDs, current page, selected product, active category, chosen materials, maximum price, sort mode, search query and stock-only filter. It begins in memory when the page loads. Since there is no local storage or server persistence, refreshing the page resets it.

Cart entries store product IDs and quantities, rather than duplicating every product field. When a screen needs a name or price, it looks that ID up in `BeejamData.products`. This avoids inconsistent copies of product information.

4.3 Initialization — lines 665–676

The `DOMContentLoaded` event fires once the HTML has been parsed. Its callback initializes navigation, renders category and featured grids, prepares shop filter choices, renders the catalogue, updates the cart badge, attaches interface listeners and draws the initial cart and checkout summaries. This ordering ensures that queried elements are present before the code tries to update them.

5. JavaScript: functions by responsibility

Function / lines	What it does	Key idea to explain
`initNavbar()` 678–699 `switchView()` 701–730	Attaches navigation listeners; hides all page views and activates the requested one; updates selected nav styling and refreshes relevant screens.	Single-page navigation changes CSS classes instead of loading a new document.
`renderHomeCategories()` 732–744 `renderHomeFeatured()` 746–753 `createProductCardMarkup()` 755–793	Builds category cards and product cards as HTML strings from data. Product cards include the price/rating and actions.	Rendering means converting data objects into visible markup. Card markup is reused across screens.
`renderShopCategoryChips()` 795–822 `onCategoryRadioChange()` 824–827 `filterByCategory()` 829–833	Creates category filter controls and updates the active category when the user chooses one.	The selected filter lives in state; a render redraws the results.
`renderShopProducts()` 835–885	Filters products by category, material, maximum price and search text; sorts by selected mode; updates count and renders either cards or an empty message.	Array `filter()` chooses matching products; `sort()` orders the resulting list.
`resetShopFilters()` 887–910	Resets filter state and visible inputs to defaults, then rerenders.	State and controls must be reset together so the display agrees with the data.
`openProductDetail()` 912–979 `setMainPDPImage()` 981–988 `switchPDPTab()` 990–1000	Finds the selected product; fills title, price, images, thumbnails, specs and related cards; switches image and informational tab when requested.	Product ID is the link between a card click and the chosen catalog record.
`addToCart()` 1002–1014 `updateCartQty()` 1016–1027 `removeFromCart()` 1029–1034	Adds a product or increments its quantity; adjusts an item quantity; removes an item when requested or quantity reaches zero.	The cart array is the source of truth; totals and badges are derived from it.
`updateCartCounters()` 1036–1041 `getCartTotals()` 1043–1050 `fmtINR()` 1052–1054	Calculates total unit count, subtotal, delivery and total; formats amounts in Indian rupees.	Delivery is free when subtotal is zero or at least ₹499; otherwise it is ₹40.
`renderCartView()` 1056–1116 `renderCheckoutSummary()` 1118–1150	Rebuilds cart lines and checkout summary from cart state, and writes current totals into their summary elements.	Rendering after every cart change keeps repeated totals in sync.

`goToCheckout()` 1152–1158 `CHECKOUT_RULES` 1160–1167 `setFieldError()` 1169–1175 `validateCheckoutForm()` 1177–1188	Blocks checkout for an empty cart; declares and checks name, email, phone, address, city and pincode; displays inline field errors and focuses the first invalid field.	Each rule is a test function; the first invalid field gets focus to guide correction.
`processCheckoutOrder()` 1190–1214	Checks that cart and form are valid, copies checkout values into `state.checkoutInfo`, then opens the payment overlay.	Validation happens before the customer details are saved for later screens.
Input listener 1216–1229 `setupEventListeners()` 1231–1315	Restricts phone/pincode input to digits and attaches search, sort, price, material, quantity, cart and wishlist interactions.	Event listeners respond to user actions and update state before calling render functions.
`toggleWishlist()` 1317–1333 `showToast()` 1335–1353	Adds/removes a product ID from the wishlist and updates its badge; creates a short-lived notification.	Wishlist membership is stored as IDs, and notifications give feedback without navigating away.
`openLogin()` 1355–1361 `closeLogin()` 1363–1369 `handleLogin()` 1371–1384 `togglePasswordVisibility()` 1386–1391	Shows/hides login overlay, imitates a brief sign-in action and changes password input visibility.	This is presentation behavior; it does not verify credentials with a server.
Account listener 1393–1403 `renderDeliverTo()` 1407–1421	Wires the account button and displays the saved delivery information in payment UI.	Checkout information is reused across the next step.
`openPayment()` 1423–1441 `closePayment()` 1443–1453	Checks for checkout info, fills delivery/order details, opens the modal, locks page scrolling and starts a timer; closing reverses the overlay and timer.	Overlay visibility and timer lifecycle are managed together.
`renderPaymentItems()` 1455–1474 `updatePaymentTotals()` 1476–1488	Draws item rows and puts the same cart totals into QR, UPI and card sections.	One calculation function feeds multiple payment displays.
`startPaymentTimer()` 1490–1509 `switchPayTab()` 1511–1518 `copyUpiId()` 1520–1526	Counts down from 10 minutes, changes selected payment panel, and attempts to copy the demo UPI ID.	`setInterval` repeats a function once per second; the clipboard API can fail and is caught.
`confirmPayment()` 1528–1551 `renderOrderDetails()` 1553–1595	Creates a local order record, closes payment, empties cart and shows the order detail view; fills order and customer details into the page.	The order number is generated from the current timestamp; success is simulated in the browser.
`sendUpiRequest()` 1597–1606 `handleCardPayment()` 1608–1617	Performs basic input checks, shows a delay/processing message and calls the local success function.	No UPI network request or card processor is called.
`formatCardNumber()` 1619–1628 `formatExpiry()` 1630–1644	Removes non-digits, limits input length and formats text for the card preview.	These helpers format input only; formatting is not payment security.

`toggleMobileNav()` 1646–1659 `closeMobileNav()` 1661–1672 `toggleMobileFilters()` 1674–1690 Escape listener 1691–1695	Opens/closes mobile navigation and filters, manages overlay/scroll lock/expanded state, and closes the drawer with Escape.	Responsive controls are coordinated by adding/removing CSS classes.

6. CSS: appearance and responsive behavior

6.1 Tokens and base rules — lines 1–145

`:root` defines reusable CSS custom properties for the botanical color palette, corner radii, font stacks and transition. For example, `var(--color-forest)` reuses the forest green value throughout the stylesheet. Typography helper classes represent IBM Plex Serif weights/styles. The universal reset applies border-box sizing and clears default margins/padding. The body sets the font, background gradient, base text color, line height and outer spacing.

6.2 Shared visual building blocks

CSS region	What it styles
Lines 147–270	Sticky site header, logo, desktop navigation, action buttons and count badges.
Lines 272–339	Page view visibility/entry animation and common filled/outline buttons.
Lines 340–784	Home hero, section headings, category cards, product card grid and promotional banner.
Lines 785–1066	Shop columns, filter controls, product listing, sorting/pagination and empty state.
Lines 1067–1436	Product gallery, thumbnails, detail content, quantity/actions and tab panels.
Lines 1437–1634	Cart layout, item rows and order summary.
Lines 1635 onward	Checkout summary and later page/overlay styling. Subsequent rules refine success, login, payment and order details.

CSS selectors choose elements by tag, class, ID, state or relationship. For example, `.page-view` hides page sections, and `.page-view.active-view` displays the active one. Pseudo-classes such as `:hover` add pointer feedback; pseudo-elements such as `::after` draw decoration without extra HTML. Flexbox and grid arrange cards and columns. Transitions/animations soften changes.

6.3 Responsive and later component rules

- `@media (max-width: 1024px)` around line 2310 adapts the layout for medium-width screens.
- `@media (max-width: 768px)` around lines 1930 and 2352 changes navigation, home, shop, product detail, cart, checkout, sustainability, login and payment arrangements for phones/tablets.
- `@media (max-width: 480px)` around lines 1941 and 2691 tightens layouts for smaller phones.
- `@media (max-width: 600px)` around lines 3841 and 3857 adds specific payment-overlay fit fixes.
- `@media (max-width: 360px)` around line 3944 handles very narrow screens.
- Login and payment sections have dedicated visual rules (roughly lines 2814–3727), including fields, spinner, payment tabs, QR presentation, UPI panel, card preview and summary.

Media queries apply only when the viewport matches their width condition. The mobile CSS can change columns to stacked layouts, hide or reveal controls, and allow the sidebar to open as a drawer. Some later rules are overrides: when selectors have equal specificity, a later matching declaration can replace an earlier value.

7. How a complete shopping visit works

1. **Startup:** HTML is parsed; the initial home view is visible; JavaScript initializes and renders catalog-based sections.
2. **Browse/search:** The visitor opens Shop. Search/filter/sort events update fields in `state` and call `renderShopProducts()`.
3. **Choose a product:** Clicking a card passes its product ID into `openProductDetail()`. The detail view is filled from the matching object.
4. **Add to cart:** The cart array is updated, then badge, item list and summaries are redrawn.
5. **Checkout:** Empty carts are blocked. The customer fills the form; custom validation shows errors and saves valid values.
6. **Payment demo:** Payment overlay displays cart, address and amount. Tab selection changes the panel. UPI/card actions show a wait message then invoke the demo completion.
7. **Confirmation:** A local order snapshot is created, cart is cleared, and the order details screen is rendered.

Data flow sketch

```
BeejamData (catalog records)
          | lookup by product ID
          v
state.cart = [{ id, qty }, ...]  ---> getCartTotals() ---> cart / checkout / payment amounts
          |                                    |
          +---- render functions ------------+----> HTML elements on screen
User action ---> update state ---> rerender the affected screen
```

8. Concepts used (plain-language glossary)

Term	Meaning in this project
DOM	The browser's in-memory representation of the HTML. JavaScript reads and updates it.
Event listener	A function that runs after an event such as a click, input, key press or page-ready event.
State	Current changing app data: current page, cart contents, filters and customer details.
Render	Build or update visible HTML using the current data.
Template literal	A JavaScript string written with backticks that can insert values such as `${product.name}`.
Array methods	`map()` transforms each item, `filter()` keeps matches, `find()` finds one item, and `reduce()` combines values into a total.
CSS class toggle	Add or remove a class to switch a visual state, such as selected tab or open drawer.
Responsive design	CSS changes layout based on screen width to keep the interface usable on mobile and desktop.

9. Honest limitations and classroom observations

- **No persistence:** cart, wishlist and order state are ordinary JavaScript memory. Reloading resets them.
- **No backend:** no database/API is shown in these files; product information is hard-coded in JavaScript.
- **Payment is a demo:** the UPI and card flows use basic client-side checks, delays and a local success function. They do not charge money. Never enter real card information.
- **Login is a demo:** there is no credential verification or account session.
- **Security lesson:** real orders, prices and payment authorization must be verified on a trusted server. Client-side code can be edited by a visitor.
- **External assets:** some product/category photos load from Unsplash, and the font comes from Google Fonts. Those external resources need a network connection; local images live in `assets/`.
- **Static interface detail:** visual pagination and some stock/review information do not necessarily represent a real data service.

> **Presentation accuracy:** describe the result as an interactive storefront prototype or front-end demo, not a live e-commerce website with real payment processing.

10. Short presentation script

“Our project is BEEJAM, a sustainable-products storefront prototype. The HTML file defines all the screens and controls. The CSS file creates the green visual theme and adapts the layout to different screen sizes. The JavaScript file stores sample product data and the current shopping state. It renders product cards, applies search and filters, manages cart quantities and calculates delivery. Checkout details are checked in the browser, then the payment screen demonstrates QR, UPI and card options. After a simulated payment, it creates an order summary. The current project is a front-end prototype: cart state is not saved after refresh, login is not connected to accounts, and payment is not connected to a real gateway.”

Likely teacher questions

Question	Clear answer
Why three files?	They separate structure (HTML), presentation (CSS) and behavior/data (JavaScript), making the project easier to organize.
How does the cart know product details?	It stores product IDs and quantities, then finds the matching product object in the catalog when it needs names and prices.
How is the total calculated?	For each cart entry, price times quantity is added to the subtotal. Delivery is ₹40 below ₹499 and free at ₹499 or more. Total is subtotal plus delivery.
How does the site change pages without reloading?	Each screen is a section. JavaScript removes the active class from all sections and adds it to the requested section; CSS hides inactive sections.
What happens if the page refreshes?	The in-memory JavaScript state is recreated, so the cart, wishlist and current order reset.
Is the payment real?	No. It is a front-end simulation and does not contact a payment service.

11. Useful source locations to show while presenting

Demonstration point	Open this source area
Fonts, metadata and stylesheet link	`index.html` lines 1–18
App navigation and mobile drawer	`index.html` lines 19–113
Catalog objects	`script.js` lines 6–647
State and initialization	`script.js` lines 648–676
Filtering/sorting	`script.js` lines 835–910
Cart totals and formatting	`script.js` lines 1002–1054
Checkout rules	`script.js` lines 1152–1229
Simulated order completion	`script.js` lines 1528–1617
Theme variables	`style.css` lines 1–35
Mobile layout changes	`style.css` lines 1930 onward and 2352 onward

End of guide. This explanation groups related source lines into understandable features so the entire project can be presented without reading thousands of lines aloud.